UNIT 4: INHERITANCE AND ABSTRACTION

- 1 — Title

UNIT 4

INHERITANCE AND ABSTRACTION

Inheritance • Overriding • Polymorphism • super • Abstract Classes • protected • final

---

- 2 — Unit Overview

Topics

- Class Inheritance
- Method Overriding
- Runtime Polymorphism
- `super`
- Superclass Constructor
- Abstraction
- Abstract Classes
- `protected`
- `final`

Industry Applications

- Payment systems
- HR systems
- Data-processing applications
- Model management
- Transport systems
- Enterprise applications

- 3 — Learning Outcomes

Students will be able to:

1. Implement class inheritance.
2. Override inherited methods.
3. Explain runtime polymorphism.
4. Use `super`.
5. Invoke superclass constructors.

6. Design abstract classes.
7. Apply `protected`.
8. Use `final`.
9. Develop reusable inheritance-based applications.

SECTION 1

INHERITANCE

- 4 — What is Inheritance?

Inheritance allows a class to acquire properties and behaviour from another class.

Syntax

```
class Child extends Parent {
}
```

Example

```
Vehicle
   |
   +---- Car
   |
   +---- Bike
```

Benefit

Code reuse + specialization

- 5 — Basic Inheritance

```
class Vehicle {

	void start() {

	System.out.println(
		"Vehicle starts"
	);
	}
}

class Car extends Vehicle {

	void openBoot() {

	System.out.println(
		"Boot opened"
	);
	}
```

```
}
```

- 6 — Using Inheritance

```
public class Main {

	public static void main(String[] args) {

	Car c = new Car();

	c.start();

	c.openBoot();
	}
}
```

Output

```
Vehicle starts
Boot opened
```

SECTION 2

METHOD OVERRIDING

- 7 — Method Overriding

When a subclass provides a new implementation of an inherited method, it is called **method overriding**.

```
class Vehicle {

	void start() {
	System.out.println(
		"Generic start"
	);
	}
}

class Car extends Vehicle {

	@Override
	void start() {

	System.out.println(
		"Car starts"
	);
	}
}
```

- 8 — Why @Override?

`@Override` tells the compiler:

This method is intended to override a superclass method.

Benefits

- Detects spelling errors
- Detects incorrect method signatures
- Improves readability
- Makes intention explicit

SECTION 3

POLYMORPHISM

- 9 — What is Polymorphism?

Polymorphism means:

One interface/reference can represent different forms of behaviour.

Example

```
Vehicle v;

v = new Car();
v.start();

v = new Bike();
v.start();
```

Same reference type.

Different runtime behaviour.

- 10 — Runtime Polymorphism

```
class Employee {

	double salary() {
	return 0;
	}
}

class FullTime extends Employee {

	@Override
```

```
	double salary() {
	return 60000;
	}
}

class Intern extends Employee {

	@Override
	double salary() {
	return 15000;
	}
}
```

- 11 — Polymorphic Collection

```
Employee[] team = {

	new FullTime(),
	new Intern()
};

for(Employee e : team) {

	System.out.println(
	e.salary()
	);
}
```

Key Idea

The method executed depends on the **actual object at runtime**.

SECTION 4

SUPER KEYWORD

- 12 — super Keyword

`super` refers to the immediate superclass.

Uses

1. Call superclass constructor
2. Call superclass method
3. Access superclass field where permitted

- 13 — Superclass Constructor

```
class Person {
```

```
    String name;

      Person(String name) {

      this.name = name;
      }
}

class Student extends Person {

      int rollNo;

      Student(String name, int rollNo) {

      super(name);

      this.rollNo = rollNo;
      }
}
```

Important

`super(...)` must be the first statement in a constructor.

- 14 — Calling Superclass Method

```
class Person {

      void display() {

      System.out.println(
            "Person information"
      );
      }
}

class Student extends Person {

      @Override
      void display() {

      super.display();

      System.out.println(
            "Student information"
      );
      }
}
```

- 15 — Topic Quiz: Inheritance

Question

Which keyword is used to create a subclass in Java?

A. implements
B. extends
C. inherits
D. subclass

Answer

B — extends

- 16 — Topic Quiz: super

Which keyword is used to invoke the constructor of the immediate superclass?

A. this
B. parent
C. super
D. base

Answer

C — super

SECTION 5

ABSTRACTION

- 17 — What is Abstraction?

Abstraction means:

Showing essential features while hiding implementation details.

Example

When using an ATM:

You see:

```
Withdraw
Deposit
Balance
```

You do not see the internal banking implementation.

- 18 — Abstract Class

An abstract class:

- Cannot be instantiated directly.
- May contain abstract methods.
- May contain concrete methods.
- May contain constructors.
- Can contain variables and methods.

- 19 — Abstract Class Example

```
abstract class Shape {

	abstract double area();

	void printType() {

	System.out.println(
		"This is a shape"
	);
	}
}
```

- 20 — Concrete Subclass

```
class Circle extends Shape {

	private double radius;

	Circle(double radius) {

	this.radius = radius;
	}

	@Override
	double area() {

	return Math.PI *
			radius *
			radius;
	}
}
```

SECTION 6

PROTECTED

- 21 — protected Access

`protected` provides:

- Access within the same package
- Access to subclasses under Java's protected rules

Example

```
class Account {

	protected double balance;
}

class SavingsAccount
	extends Account {

	void addInterest(double rate) {

	balance +=
		balance * rate;
	}
}
```

Design Tip

Use `protected` carefully; prefer stronger encapsulation where possible.

SECTION 7

FINAL KEYWORD

- 22 — final Keyword

`final` can be applied to:

Variable

Cannot be reassigned.

Method

Cannot be overridden.

Class

Cannot be extended.

- 23 — final Method

```
class Audit {
```

```
	final void log() {

	System.out.println(
		"Audit completed"
	);
	}
}
```

A subclass cannot override:

```
void log()
```

- 24 — final Class

```
final class SecurityUtility {

	static void authenticate() {

	System.out.println(
		"Authentication"
	);
	}
}
```

Cannot write:

```
class Child
	extends SecurityUtility {
}
```

- 25 — Topic Quiz: Abstraction

Which statement about an abstract class is correct?

A. It must always be instantiated directly
B. It cannot contain concrete methods
C. It cannot be instantiated directly
D. It cannot have constructors

Answer

C — It cannot be instantiated directly

- 26 — Topic Quiz: final

Which keyword prevents a method from being overridden?

A. static
B. private
C. final
D. const

Answer

C — final

-27- Revision Quiz

Which keyword is used to create a subclass in Java?

A. implements
B. extends
C. inherits
D. subclass

Answer: B

-28-

Which statement about an abstract class is correct?

A. It must always be instantiated directly
B. It cannot contain concrete methods
C. It cannot be instantiated directly
D. It cannot have constructors

Answer: C

-29-

Which keyword prevents a method from being overridden?

A. static
B. private
C. final
D. const

Answer: C

-30-

Which statement about an interface is correct?

A. A class can implement only one interface
B. An interface defines a contract that implementing classes follow
C. Interfaces can be instantiated directly
D. Interfaces require all methods to be private

Answer: B

-31-

Which access level permits access within the same package and supports subclass access under Java's protected rules?

A. public
B. private
C. protected
D. final

Answer: C

-32-
Which statement best describes a Java class?

A. A runtime instance of another class
B. A blueprint that defines data and behavior
C. A package containing only methods
D. A primitive data type

-33-
Which feature allows two methods in the same class to have the same name but different parameter lists?

A. Method overriding
B. Encapsulation
C. Method overloading
D. Abstraction

Answer: C

-34-
Which access specifier provides the most restricted direct access to a class member?

A. public
B. protected
C. private
D. default

Answer: C

-35-
Which keyword is used to invoke the constructor of the immediate superclass?

A. this
B. parent
C. super
D. base

Answer: C

-36-
Which OOP concept allows a superclass reference to refer to objects of different subclasses?

A. Encapsulation
B. Runtime polymorphism
C. Constructor chaining only
D. Data hiding

Answer: B

-38-

Which OOP concept allows a superclass reference to refer to objects of different subclasses?

A. Encapsulation
B. Runtime polymorphism
C. Constructor chaining only
D. Data hiding

Answer

B — Runtime polymorphism